Mini Project Report

on

BondHealth

Submitted by

Mariyam Rahim (20223068)
Pillai Anjita (20223079)
Sneha Vijayan (20223099)
Soya R F (20223101)
Thara Asharaf (20223108)

In partial fulfilment of the requirements for the award of degree of Bachelor of Technology in Computer Science and Engineering.

DIVISION OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF ENGINEERING
COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY

APRIL 2026

DIVISION OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF ENGINEERING
COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY

CERTIFICATE

Certified that this is a Mini Project Report titled

BondHealth

Submitted by

Mariyam Rahim (20223068)
Pillai Anjita (20223079)
Sneha Vijayan (20223099)
Soya R F (20223101)
Thara Asharaf (20223108)

of VI Semester, Computer Science and Engineering in the year 2024 in partial fulfillment requirements for the award of degree of Bachelor of Technology in Computer Science and Engineering of Cochin University of Science and Technology.

Mr. Pramod Pavithran	Ms. Sheena S	Ms. Sheena S
Head of Division	**Project Coordinator**	**Project Guide**

DECLARATION

We, Mariyam Rahim, Pillai Anjita, Sneha Vijayan, Soya R F and Thara Asharaf, hereby declare that the project titled **"BondHealth"** is a bona fide work carried out by us during the year 2026, under the guidance of **Mrs. Sheena S**, Assistant Professor, School of Engineering, CUSAT. We further declare that this project has not been submitted, either in part or full, for the award of any degree, diploma, associateship, fellowship, or any other academic recognition in any university or institution.

ABSTRACT

BondHealth is a comprehensive digital healthcare ecosystem designed to unify patient care across multiple healthcare providers. This project addresses the critical challenge of fragmented medical records by creating a centralized platform where patients' medical history, appointments, and lab results are securely accessible across all connected hospitals.

The system features role-based dashboards for patients, doctors, hospital administrators, and lab technicians. Key functionalities include appointment scheduling, electronic medical records management, lab report integration, prescription management, and a public review system. Built with Node.js and Express on the backend, PostgreSQL for data persistence, and vanilla JavaScript with Tailwind CSS for the frontend, BondHealth demonstrates a production-ready healthcare management solution.

The platform implements robust authentication using JWT tokens, role-based access control, and comprehensive API endpoints for all operations. With features like real-time appointment management, leave tracking for doctors, and a dynamic review system, BondHealth provides a seamless experience for all stakeholders in the healthcare delivery chain.

Keywords: Healthcare Management System, Electronic Medical Records, Patient Portal, Telemedicine, Hospital Management, Node.js, PostgreSQL, JWT Authentication, Role-Based Access Control

Contents

List of Figures

List of Tables

Chapter 1

Introduction

The healthcare industry today faces significant challenges in managing and accessing patient medical information. In the current system, patient data is often scattered across multiple hospitals and healthcare providers, creating fragmented medical histories that are difficult to access when needed. This fragmentation leads to several critical issues: patients become dependent on physical reports, undergo repeated diagnostic tests, face limited options for remote consultations, and encounter difficulties in obtaining certain medications, especially rare ones. Additionally, hospitals operate without a common digital platform to efficiently manage patient data and streamline interactions between healthcare providers and patients.

The digital transformation of healthcare has become increasingly important in recent years, with the COVID-19 pandemic highlighting the urgent need for remote healthcare solutions. Patients, particularly the elderly and those with mobility issues, require accessible healthcare services that do not always necessitate physical hospital visits. Furthermore, the need for a centralized, secure system to store and access medical records has become paramount in ensuring continuity of care and improving healthcare outcomes.

1.1 Problem Statement

The existing healthcare ecosystem suffers from the following key problems:

- **Fragmented Medical Records**: Patient medical information is distributed across different hospitals, making it difficult to obtain complete medical histories when required for diagnosis or treatment.
- **Dependency on Physical Reports**: Patients must carry physical copies of their medical reports, which can be lost, damaged, or forgotten during emergencies.
- **Repeated Diagnostic Tests**: Due to inaccessible medical histories, patients often undergo redundant tests, increasing healthcare costs and inconvenience.
- **Limited Remote Consultation Options**: Many patients, especially the elderly and those in remote areas, struggle to access healthcare services that require physical presence.
- **Difficulty in Obtaining Medications**: Patients face challenges in locating and purchasing rare or specific medicines that may only be available at certain hospital pharmacies.

- **Lack of a Common Digital Platform**: Hospitals lack a unified digital interface to efficiently manage patient data, appointments, consultations, and billing processes.

1.2 Objectives

The primary objectives of the BondHealth project are:

1. **Develop a Centralized Digital Platform**: Create a web-based healthcare platform that connects hospitals and patients within a specific geographical area, functioning as a digital interface for healthcare services.

2. **Implement Digital Patient Identities**: Provide unique, secure digital IDs to patients (similar to DigiLocker) for storing and accessing medical records including consultation history, diagnostic reports, and medical scans.

3. **Enable Hospital Discovery**: Display registered hospitals along with available departments and doctor information, allowing patients to make informed decisions about their healthcare providers.

4. **Facilitate Remote Consultations**: Support scheduled chat and video call consultations between patients and doctors, making healthcare accessible to those unable to visit hospitals in person.

5. **Integrate Pharmacy Services**: Enable online purchase of medicines, including rare medications, from hospital pharmacies with electronic billing for transparent, paperless transactions.

6. **Ensure Security and Privacy**: Implement robust security measures to protect sensitive patient data and ensure compliance with healthcare data protection standards.

1.3 Scope of the Project

The scope of BondHealth encompasses the following aspects:

1.3.1 Geographical Scope

The platform initially targets a specific geographical area, connecting hospitals and patients within that region. This focused approach allows for better management, testing, and refinement of the system before potential expansion to wider areas.

1.3.2 Functional Scope

- **Patient Registration and Authentication**: Secure registration process assigning unique patient IDs with robust authentication mechanisms.
- **Medical Records Management**: Digital storage and retrieval of consultation histories, diagnostic reports, and medical scans.
- **Hospital and Doctor Directory**: Comprehensive listing of registered hospitals with department-wise doctor information.
- **Teleconsultation Services**: Scheduling and conducting remote consultations through chat and video calls.
- **Online Pharmacy**: E-commerce functionality for medicine purchases from hospital pharmacies.
- **Electronic Billing**: Digital invoice generation and payment processing for consultations and medicine purchases.

1.3.3 Technical Scope

The project involves developing a responsive web application using modern web technologies, implementing secure database management for medical records, integrating video calling APIs for teleconsultation, and ensuring data encryption and privacy compliance.

Chapter 2

SYSTEM STUDY

The system study phase involves understanding the existing healthcare management challenges and analyzing the requirements for BondHealth. This chapter covers the problem definition, feasibility analysis, and detailed requirements gathering for the proposed system.

2.1 Existing System

2.1.1 Current Healthcare Challenges

- **Fragmented Medical Records:** Patient data is scattered across different hospitals, clinics, and laboratories with no central access.
- **Manual Appointment Management:** Many healthcare facilities still rely on phone calls and paper-based appointment scheduling.
- **Delayed Lab Results:** Test reports take days to reach doctors and patients due to manual delivery systems.
- **Lack of Patient Engagement:** Patients have limited access to their own health information and medical history.
- **Administrative Inefficiency:** Hospital staff spend excessive time on paperwork and manual data entry.

2.1.2 Disadvantages of Existing System

- No centralized patient health record system
- Increased wait times for appointments and results
- Higher chances of medical errors due to missing information
- Poor communication between different healthcare providers
- Limited scalability for growing healthcare networks

2.2 Proposed System

2.2.1 System Overview

BondHealth is a web-based healthcare management platform that connects patients, doctors, hospitals, and laboratories on a single unified system. The platform provides:

- **Centralized Patient Records:** Complete medical history accessible to authorized healthcare providers
- **Online Appointment System:** 24/7 appointment booking with real-time availability
- **Digital Lab Reports:** Instant sharing of test results between labs, doctors, and patients
- **Role-Based Dashboards:** Customized interfaces for different user types
- **Secure Authentication:** JWT-based authentication with role-based access control

2.2.2 Advantages of Proposed System

- Seamless access to patient medical history across all connected hospitals
- Reduced wait times through online appointment scheduling
- Instant delivery of lab reports to doctors and patients
- Enhanced patient engagement through personal health portal
- Improved administrative efficiency with automated workflows
- Scalable architecture for future expansion

2.3 Feasibility Study

2.3.1 Technical Feasibility

The system is technically feasible as it uses:

- **Node.js & Express:** Proven backend technology for building scalable web applications
- **PostgreSQL:** Robust relational database with excellent performance and data integrity
- **JWT Authentication:** Industry standard for secure API authentication
- **Responsive Design:** Works seamlessly on desktops, tablets, and mobile devices

2.3.2 Operational Feasibility

The system is operationally feasible because:

- User-friendly interfaces require minimal training for staff
- Role-based dashboards simplify daily tasks for each user type
- Automated notifications reduce manual follow-up work
- Centralized records eliminate redundant data entry

2.3.3 Economic Feasibility

The system offers significant economic benefits:

- Reduced administrative costs through automation
- Elimination of paper-based systems and storage
- Improved staff productivity and efficiency
- Scalable architecture grows with organizational needs
- Open-source technology stack minimizes licensing costs

2.4 Requirements Analysis

2.4.1 Functional Requirements

2.4.1.1 Patient Module

- User registration and profile management
- View and manage appointments
- Book new appointments with available doctors
- Access medical reports and prescriptions
- Update personal and emergency contact information
- Write and view public reviews

2.4.1.2 Doctor Module

- View today's appointment schedule
- Access patient medical history
- Review and add findings to lab reports
- Manage availability and leave requests
- Communicate with patients

2.4.1.3 Hospital Admin Module

- Register new doctors and staff
- Manage medicine inventory
- Add laboratory facilities
- View hospital statistics and reports
- Manage doctor leave schedules

2.4.1.4 Lab Technician Module

- Upload and manage lab reports
- Update report status (pending/completed)
- Share results with doctors and patients
- View pending test requests

2.4.2 Non-Functional Requirements

2.4.2.1 Performance Requirements

- Page load time should be less than 3 seconds
- System should handle 1000+ concurrent users
- Database queries should execute within 500ms
- API response time under 200ms for standard operations

2.4.2.2 Security Requirements

- All passwords must be hashed using bcrypt
- JWT tokens for API authentication with 7-day expiration
- Role-based access control for all operations
- Session management with active session tracking
- Secure cookie handling with HttpOnly flag

2.4.2.3 Usability Requirements

- Intuitive interface requiring minimal training
- Responsive design for all device sizes
- Clear error messages and user feedback
- Consistent navigation across all modules

2.4.2.4 Reliability Requirements

- 99.9% uptime during business hours
- Data backup and recovery procedures
- Transaction integrity for critical operations
- Graceful error handling without data loss

2.5 System Architecture Overview

Three-Tier Architecture

- **Presentation Layer:** HTML, CSS, JavaScript, Tailwind CSS
- **Application Layer:** Node.js, Express.js
- **Data Layer:** PostgreSQL Database

Figure 2.1: BondHealth Three-Tier Architecture

The BondHealth system follows a three-tier architecture:

1. **Client Tier:** Web browsers render HTML pages with dynamic content fetched via API calls

2. **Application Tier:** Express.js server handles HTTP requests, implements business logic, and manages authentication
3. **Data Tier:** PostgreSQL database stores all application data with proper relationships and constraints

2.6 System Modules

2.6.1 Authentication Module

Handles user registration, login, and session management. Implements:

- JWT token generation and verification
- Password hashing with bcrypt
- Role-based access control
- Session tracking and management

2.6.2 Patient Management Module

Manages all patient-related operations:

- Profile creation and updates
- Medical history tracking
- Appointment scheduling
- Report and prescription access

2.6.3 Appointment Management Module

Handles the complete appointment lifecycle:

- Real-time availability checking
- Appointment booking, rescheduling, and cancellation
- Conflict detection and prevention
- Status tracking and notifications

2.6.4 Reporting Module

Manages lab reports and medical documents:

- Report upload and storage
- Status tracking (pending/reviewed)
- Secure sharing between stakeholders
- Historical data access

2.7 User Roles and Permissions

Role	**Permissions**
Patient	View own records, book appointments, access reports
Doctor	View appointments, access patient history, review reports
Hospital Admin	Manage doctors, staff, inventory, view analytics
Lab Technician	Upload reports, update status, share results

Table 2.1: User Roles and Access Levels

2.8 Summary

The system study phase has established a clear understanding of the existing healthcare challenges and defined comprehensive requirements for BondHealth. The proposed system addresses critical gaps in patient data management, appointment scheduling, and report sharing. With its modular architecture, role-based access control, and focus on usability, BondHealth is well-positioned to transform healthcare delivery in connected facilities.

Chapter 3

Technologies Used

3.1 Introduction

BondHealth leverages modern web technologies to create a robust, scalable, and user-friendly healthcare management system. This chapter discusses the various technologies, frameworks, and tools employed in the development of the project. The technology stack was carefully selected to ensure optimal performance, security, and maintainability while providing an excellent user experience across all devices.

Component	**Technologies Used**
Frontend	HTML5, Tailwind CSS, Font Awesome
Backend	Node.js v18.17.0, Express.js v4.18.2
Database	PostgreSQL v15, Neon Serverless
Authentication	JWT, bcryptjs v2.4.3
Development Tools	VS Code, Git, GitHub, npm
Deployment	Environment Variables, SSL/TLS

Table 3.1: BondHealth Technology Stack Overview

3.2 Frontend Technologies

3.2.1 HTML5

HTML5 provides the structural foundation for all web pages in BondHealth. It offers semantic elements that improve accessibility and SEO. Key HTML5 features utilized include:

- **Semantic tags**: `<header>`, `<nav>`, `<section>`, `<article>`, `<footer>` for better document structure
- **Form validation**: Built-in validation attributes like `required`, `pattern`, and `type="email"` for enhanced user experience
- **Local storage**: Client-side data persistence for saving user preferences and session state
- **Responsive images**: `<picture>` element and `srcset` attributes for optimal image delivery across devices

3.2.2 Tailwind CSS

Tailwind CSS v3.3.0 is a utility-first CSS framework that enables rapid UI development. Its advantages in BondHealth include:

- **Utility-first approach**: Pre-built classes for margins, padding, colors, flexbox, and grid systems
- **Responsive design**: Built-in breakpoints (sm, md, lg, xl, 2xl) for mobile, tablet, and desktop views
- **Customization**: Easy theming with the `tailwind.config.js` file for brand colors and fonts
- **Performance**: Unused styles are purged in production, reducing CSS bundle size by approximately 70%

Example of Tailwind CSS usage in the patient dashboard header:

Listing 3.1: Patient Dashboard Header from BondHealth

```
<div class="white-card␣rounded-2xl␣p-6␣mb-6␣fade-in">
  <div class="flex␣flex-col␣md:flex-row␣justify-between␣items-
     center">
    <div class="flex␣items-center␣space-x-4␣mb-4␣md:mb-0">
      <div class="w-16␣h-16␣cyan-bg␣rounded-full␣flex␣items-
         center␣justify-center␣text-2xl␣font-bold␣text-white">
        ${patient.name.charAt(0)}
      </div>
      <div>
        <h1 class="text-3xl␣font-bold␣cyan-text">Welcome back, <
           span class="text-gray-800">${patient.name}</span></h1>
        <p class="text-gray-600">Your health, our priority</p>
      </div>
    </div>

    <div class="flex␣space-x-4">
      <div class="cyan-light␣rounded-xl␣p-4␣min-w-[120px]␣text-
         center">
        <p class="text-sm␣cyan-text␣font-medium">Patient ID</p>
        <p class="text-xl␣font-bold␣cyan-text">${patient.id}</p>
      </div>
      <button id="logoutBtn" class="bg-red-500␣hover:bg-red-600␣
         text-white␣rounded-xl␣px-6␣py-4">
        <i class="fas␣fa-sign-out-alt␣mr-2"></i>Logout
      </button>
    </div>
```

```
  </div>
</div>
```

The custom CSS classes like `cyan-bg`, `cyan-text`, and `white-card` were defined in the style section:

Listing 3.2: Custom CSS Classes for BondHealth

```
.cyan-bg {
  background: linear-gradient(135deg, #00bcd4 0%, #00acc1 100%);
}

.cyan-light {
  background: #e0f7fa;
}

.cyan-text {
  color: #006064;
}

.white-card {
  background: white;
  box-shadow: 0 10px 40px rgba(0, 188, 212, 0.1);
}
```

3.2.3 Font Awesome

Font Awesome v6.4.0 provides a comprehensive library of scalable vector icons that enhance the user interface. Icons are used throughout BondHealth for:

- **Navigation menu items**: `fa-calendar-day`, `fa-file-medical-alt`, `fa-user-friends`
- **Action buttons**: `fa-edit` (edit), `fa-trash` (delete), `fa-download` (download), `fa-share-alt` (share)
- **Status indicators**: `fa-check-circle` (success), `fa-exclamation-triangle` (warning), `fa-info-circle` (info)
- **Profile avatars**: `fa-user-md` (doctor), `fa-user-injured` (patient), `fa-hospital` (hospital)
- **Medical symbols**: `fa-stethoscope`, `fa-prescription`, `fa-vial`, `fa-heartbeat`

3.3 Backend Technologies

3.3.1 Node.js

Node.js v18.17.0 (LTS) is a JavaScript runtime built on Chrome's V8 engine. It enables server-side JavaScript execution and provides an event-driven, non-blocking I/O model.

Benefits in BondHealth include:

- **Single language**: JavaScript used for both frontend and backend, reducing context switching
- **NPM ecosystem**: Access to over 1.3 million open-source packages
- **Scalability**: Handles multiple concurrent connections efficiently with event loop architecture
- **Performance**: Fast execution for real-time applications with asynchronous I/O

3.3.2 Express.js

Express.js v4.18.2 is a minimal and flexible Node.js web application framework. Key features used:

- **Routing**: Handling HTTP requests for different endpoints (`GET`, `POST`, `PUT`, `DELETE`)
- **Middleware**: Parsing JSON, handling CORS, logging requests, error handling
- **Template rendering**: Serving dynamic HTML content through functions
- **Error handling**: Centralized error management with custom error handlers

Example Express.js route handler from BondHealth's appointment system:

Listing 3.3: Express.js Route Handler from BondHealth

```
// Get today's appointments for doctor dashboard
app.get('/api/appointments/today', async (req, res) => {
  try {
    const doctorId = req.user.doctor_id;
    const result = await query(
      `SELECT a.*, p.full_name as patient_name, p.patient_uuid
       FROM appointments a
       JOIN patients p ON a.patient_id = p.patient_id
       WHERE a.doctor_id = $1
       AND a.appointment_date = CURRENT_DATE
       ORDER BY a.appointment_time`,
      [doctorId]
    );
    res.json(result.rows);
  } catch (error) {
    console.error('Error fetching appointments:', error);
    res.status(500).json({ error: 'Failed to fetch appointments'
       });
  }
```

```
});

// Book new appointment
app.post('/api/appointments', async (req, res) => {
  try {
    const { doctor_id, appointment_date, appointment_time, reason
       , type } = req.body;
    const patient_id = req.user.patient_id;

    const result = await query(
      `INSERT INTO appointments
       (patient_id, doctor_id, appointment_date, appointment_time
          , reason, appointment_type, status)
       VALUES ($1, $2, $3, $4, $5, $6, 'confirmed')
       RETURNING *`,
      [patient_id, doctor_id, appointment_date, appointment_time,
          reason, type]
    );

    res.status(201).json(result.rows[0]);
  } catch (error) {
    console.error('Error booking appointment:', error);
    res.status(500).json({ error: 'Failed to book appointment' })
       ;
  }
});
```

3.4 Database Technologies

3.4.1 PostgreSQL

PostgreSQL v15 is a powerful, open-source object-relational database system. Its features utilized in BondHealth include:

- **ACID compliance**: Ensures data integrity and reliability for medical records
- **JSON support**: Store and query JSON data for flexible schemas in lab reports
- **Foreign keys**: Maintain referential integrity between tables
- **Indexes**: Optimize query performance for large datasets
- **Transactions**: Group multiple operations into atomic units for data consistency

3.4.2 Neon Database Serverless

Neon is a serverless Postgres platform that separates storage and compute for automatic scaling. Advantages include:

- **Serverless architecture**: No infrastructure management, automatic backups
- **Autoscaling**: Handles varying workloads automatically
- **Branching**: Create database branches for development and testing
- **Point-in-time recovery**: Restore data to any point in time
- **Connection pooling**: Built-in pooling for handling many concurrent connections

The connection to Neon PostgreSQL is established using:

Listing 3.4: Database Configuration in BondHealth

```
const { Pool } = require('@neondatabase/serverless');

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: {
    require: true,
  },
});

const query = async (text, params) => {
  const start = Date.now();
  try {
    const res = await pool.query(text, params);
    const duration = Date.now() - start;
    console.log('Executed query', { text, duration, rows: res.
       rowCount });
    return res;
  } catch (error) {
    console.error('Query error:', error);
    throw error;
  }
};

module.exports = { pool, query };
```

3.4.3 Database Schema Design

The BondHealth database consists of several interconnected tables designed for healthcare data management:

Table	Description
users	Stores authentication data for all user types (patients, doctors, admins, lab technicians)
patients	Contains patient personal and medical information
doctors	Stores doctor profiles and specializations
appointments	Manages scheduling between patients and doctors
lab_reports	Stores test results and medical reports
prescriptions	Manages medication prescriptions

Table 3.2: BondHealth Database Tables

The relationships between tables are as follows:

- **users** table connects to **patients** and **doctors** via `user_id` foreign keys
- **appointments** table references both **patients** (`patient_id`) and **doctors** (`doctor_id`)
- **lab_reports** and **prescriptions** tables reference **patients** and **doctors**

3.5 Development Tools

3.5.1 Visual Studio Code

VS Code v1.84 serves as the primary integrated development environment (IDE) with extensions for:

- **LaTeX Workshop**: For writing project documentation with real-time preview
- **Prettier**: Code formatting on save for consistent style
- **ESLint**: JavaScript linting and error detection
- **GitLens**: Git integration and history visualization
- **Tailwind CSS IntelliSense**: Autocomplete for Tailwind classes

3.5.2 Git and GitHub

Version control is essential for collaborative development. Features used include:

- **Branching**: Parallel development of features with main, dev, and feature branches
- **Pull requests**: Code review and collaboration with team members
- **Issues**: Task tracking and bug reporting
- **GitHub Actions**: Continuous integration for automated testing
- **.gitignore**: Properly configured to exclude LaTeX temp files and node_modules

3.5.3 NPM (Node Package Manager)

NPM v9.6.7 manages project dependencies defined in `package.json`:

Listing 3.5: BondHealth package.json Dependencies

```
{
  "name": "bondhealth",
  "version": "1.0.0",
  "description": "Healthcare Management System",
  "main": "src/signin.js",
  "scripts": {
    "start": "node src/signin.js",
    "dev": "nodemon src/signin.js"
  },
  "dependencies": {
    "express": "^4.18.2",
    "pg": "^8.11.3",
    "@neondatabase/serverless": "^0.6.0",
    "bcryptjs": "^2.4.3",
    "jsonwebtoken": "^9.0.2",
    "dotenv": "^16.3.1",
    "helmet": "^7.1.0",
    "cors": "^2.8.5"
  },
  "devDependencies": {
    "nodemon": "^3.0.1"
  }
}
```

3.6 Supporting Technologies

3.6.1 JSON Web Tokens (JWT)

JWT is used for authentication and authorization:

- **Stateless authentication**: No server-side session storage required
- **Role-based access**: Tokens contain user role information
- **Secure transmission**: Signed tokens prevent tampering
- **Expiration**: Automatic token expiry for security
- **HTTP-only cookies**: Tokens stored in secure, HTTP-only cookies

JWT implementation in BondHealth:

Listing 3.6: JWT Implementation in BondHealth

```
// Generate JWT token
const generateToken = (user) => {
    return jwt.sign(
        {
            id: user.user_id,
            username: user.username,
            role: user.role
        },
        process.env.JWT_SECRET,
        { expiresIn: '7d' }
    );
};

// Verify JWT token middleware
const verifyToken = (req, res, next) => {
    const token = req.cookies.token;

    if (!token) {
        return res.status(401).json({ error: 'No token provided'
           });
    }

    try {
        const decoded = jwt.verify(token, process.env.JWT_SECRET)
           ;
        req.user = decoded;
        next();
    } catch (error) {
        return res.status(403).json({ error: 'Invalid or expired
           token' });
    }
};
```

3.6.2 bcryptjs

bcryptjs v2.4.3 provides password hashing for secure credential storage:

- **Salting**: Random salts prevent rainbow table attacks
- **Adaptive**: Work factor can be increased over time
- **Secure comparison**: Timing-safe comparison functions

Listing 3.7: Password Hashing in BondHealth

```
// Hash password during registration
const saltRounds = 10;
const hashedPassword = await bcrypt.hash(password, saltRounds);

// Compare password during login
const isValid = await bcrypt.compare(password, user.password_hash
   );
```

3.6.3 dotenv

dotenv loads environment variables from a `.env` file:

Listing 3.8: BondHealth Environment Variables

```
# Server Configuration
PORT=3005
NODE_ENV=development

# Database Configuration
DATABASE_URL=postgresql://bondhealth_user:password@server.neon.
   tech/bondhealth

# JWT Configuration
JWT_SECRET=bondhealth_secret_key_2024
JWT_EXPIRE=7d
```

3.7 Project Structure

BondHealth follows a modular architecture with clear separation of concerns:

```
BondHealth/
|-- src/
|   |-- db/
|   |   |-- config.js
|   |   |-- init.js
|   |   |-- seed.js
|   |-- Patient.js
|   |-- Doctor.js
|   |-- admin.js
|   |-- labs.js
|   |-- signin.js
|   |-- home.js
|-- .env
```

```
|-- .gitignore
|-- package.json
|-- README.md
```

3.8 Deployment Technologies

3.8.1 Environment Configuration

The application uses environment-specific configurations:

- **Development**: Local PostgreSQL instance with debug logging enabled
- **Testing**: Isolated test database with sample data
- **Production**: Neon serverless PostgreSQL with connection pooling and SSL

3.8.2 SSL/TLS

Secure Sockets Layer ensures encrypted communication between client and server:

- **Data encryption**: Prevents eavesdropping on sensitive medical data
- **Certificate validation**: Verifies server identity
- **HTTPS enforcement**: All production traffic forced over HTTPS

3.9 Advantages of Technology Stack

3.9.1 Performance

- Node.js non-blocking I/O handles multiple concurrent requests efficiently
- PostgreSQL indexing optimizes query performance
- Tailwind CSS purges unused styles reducing bundle size

3.9.2 Scalability

- Horizontal scaling with multiple Node.js instances
- Database connection pooling for high concurrency
- Serverless database automatically scales with load

3.9.3 Maintainability

- Modular code structure with separate files for each user role
- Comprehensive documentation with LaTeX
- Version control with Git for change tracking

3.9.4 Security

- Password hashing with bcrypt
- JWT for secure authentication
- SQL injection prevention with parameterized queries
- Helmet.js for security headers
- Environment variables for sensitive configuration

3.10 Conclusion

The technology stack chosen for BondHealth provides a solid foundation for building a secure, scalable, and maintainable healthcare management system. Each technology was selected based on its strengths and compatibility with other components, resulting in a cohesive and efficient development environment.